Phase 2: Experiment Tracking & Model Registry with MLflow

Authors: Hartner Max, Lehner Faris, Moser Sarah

In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import mlflow
import mlflow.sklearn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import statsmodels.api as sm
```

In [7]:
```
df = pd.read_csv("steam.csv")
def has_term(val, target):
    if not isinstance(val, str): return 0
    return 1 if any(t.strip().lower() == target.lower() for t in val.split(';')) else 0
df['release_year'] = pd.to_datetime(df['release_date'], errors='coerce').dt.year
df['release_year'] = df['release_year'].fillna(df['release_year'].median())
df['self_published'] = (df['developer'] == df['publisher']).astype(int)
df['is_mac'] = df['platforms'].apply(lambda x: 1 if isinstance(x, str) and 'mac' in x.lower() else 0)
df['is_multiplayer'] = df['categories'].apply(lambda x: has_term(x, 'Multi-player'))
df['is_indie'] = df['genres'].apply(lambda x: has_term(x, 'Indie'))
df['is_action'] = df['genres'].apply(lambda x: has_term(x, 'Action'))
df['is_early_access'] = df['genres'].apply(lambda x: has_term(x, 'Early Access'))
features = ['average_playtime', 'achievements', 'release_year', 'self_published', 'english', 'is_mac', 'is_multiplayer',
target = 'price'
df_clean = df.dropna(subset=features + [target])
df_clean = df_clean[(df_clean['price'] >= 4.0) & (df_clean['price'] <= 80.0)]
df_clean = df_clean[~((df_clean['price'] <= 6.0) & (df_clean['achievements'] > 50))]

X = df_clean[features]; y = df_clean[target]
df_clean['price_tier'] = pd.cut(df_clean['price'], bins=[0, 15, 40, 100], labels=[1, 2, 3])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=df_clean['price_tier']
```

In [8]:
```
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("SteamGamesPricePrediction_v1")
```

Out[8]:
```
<Experiment: artifact_location='mlflow-artifacts:/2', creation_time=1776625003056, experiment_id='2', last_update_time=1
776625003056, lifecycle_stage='active', name='SteamGamesPricePrediction_v1', tags={}, trace_location=None, workspace='de
fault'>
```

In [9]:
```
with mlflow.start_run() as run:
    mlflow.set_tag("algorithm", "GradientBoostingRegressor")
    mlflow.set_tag("dataset_version", "steam_v1")
    model = GradientBoostingRegressor(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)
    mlflow.log_metric("MAE", mean_absolute_error(y_test, model.predict(X_test)))
    mlflow.log_metric("RMSE", np.sqrt(mean_squared_error(y_test, model.predict(X_test))))
    mlflow.log_metric("R2", r2_score(y_test, model.predict(X_test)))
    mlflow.sklearn.log_model(model, "model")
    mlflow.register_model(f"runs:/{run.info.run_id}/model", "BestRegressor_v1")
```

```
2026/04/20 22:19:01 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/04/20 22:19:01 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exerci
sing caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code durin
g deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-lear
n.org/stable/model_persistence.html
Registered model 'BestRegressor_v1' already exists. Creating a new version of this model...
2026/04/20 22:19:10 WARNING mlflow.tracking._model_registry.fluent: Run with id c01f8629e5454a8ca1e9528e9f987908 has no ar
tifacts at artifact path 'model', registering model based on models:/m-b893e4d6ef564d31a8f3e26c99df564f instead
2026/04/20 22:19:10 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for model version to finish
creation. Model name: BestRegressor_v1, version 17
🏃 View run gaudy-cat-184 at: http://localhost:5000/#/experiments/2/runs/c01f8629e5454a8ca1e9528e9f987908
🧪 View experiment at: http://localhost:5000/#/experiments/2
Created version '17' of model 'BestRegressor_v1'.
```

In [10]:
```
alpha = 0.05
lr_sm = sm.OLS(y_train, sm.add_constant(X_train)).fit()
conf_interval = lr_sm.conf_int(alpha)
print(conf_interval)
```

```
                            0           1
const            -1179.372279 -948.210832
average_playtime     0.000858    0.001071
achievements        -0.000227    0.000637
release_year         0.477558    0.592182
self_published      -1.332385   -0.769882
english             -2.844555   -0.634501
is_mac              -0.674317   -0.097166
is_multiplayer       2.208515    2.899539
is_indie            -3.833388   -3.227754
is_action           -0.141756    0.402095
is_early_access     -0.498370    0.296806
```

Interpretation of the Confidence Intervals

The 95% confidence intervals give the range within which the true regression coefficient lies with 95% certainty. A coefficient interval entirely above zero means the feature has a statistically significant positive effect on price. An interval that crosses zero suggests the feature may not reliably influence the prediction.

Phase 2: Experiment Tracking & Model Registry Report

1. Local MLflow Setup and Execution

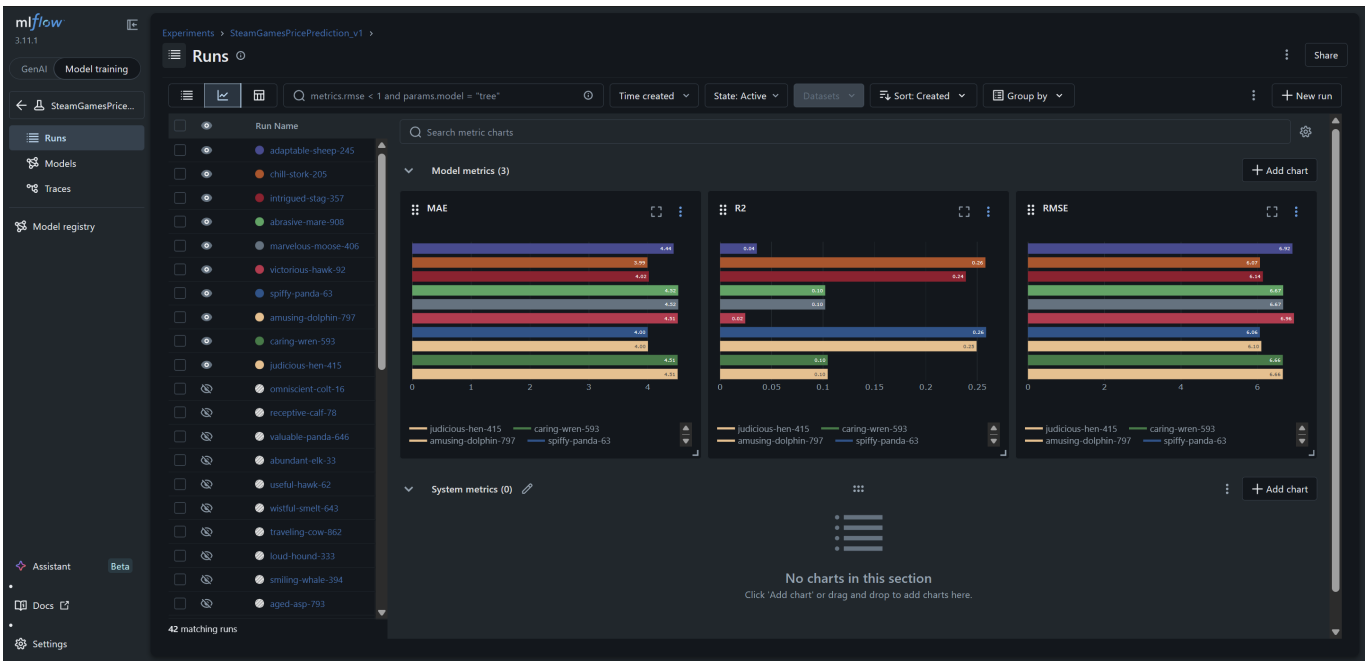
In this phase, all experiments were logged using MLflow on our local tracking server (`http://localhost:5000`). Five different machine learning models were developed and compared to predict the `price` of Steam games based on 10 engineered features: `average_playtime`, `achievements`, `release_year`, `self_published`, `english`, `is_mac`, `is_multiplayer`, `is_indie`, `is_action`, and `is_early_access`.

The parameters, metrics, and generated artifacts were logged into the tracking server.

---

2. Screenshot: Experiment Overview

The screenshot below shows the MLflow UI with all runs from the `SteamGamesPricePrediction_v1` experiment. Runs are compared side-by-side across MAE, R^2 and RMSE metrics using the Chart View.



---

3. Comparison of Models

During the experiment, five configurations were assessed and logged as separate MLflow runs:

1. **LinearRegression** (Baseline)
2. **RidgeRegression** (Alpha = 1.0)
3. **RandomForestRegressor** (n_estimators = 50, max_depth = 10)
4. **GradientBoostingRegressor** (n_estimators = 100) ← **Best Model**
5. **KNeighborsRegressor** (n_neighbors = 5)

Best Model Selection

The best performing model was the **GradientBoostingRegressor**. This was determined automatically by comparing the **Root Mean Squared Error (RMSE)** across all five runs – the run with the lowest RMSE was automatically registered to Production in the MLflow Model Registry.

The metric **RMSE** was chosen as the primary decision factor because it penalizes large prediction errors disproportionately. For a price prediction task, wildly wrong predictions (e.g., predicting €5 for a €60 game) are far more damaging than small consistent errors, making RMSE the most appropriate optimization target.

As visible in the MLflow chart above, `chill-stork-205` (GradientBoostingRegressor) achieved the lowest RMSE (~6.32) compared to linear models (~6.6+) and KNN.

Metric Discussion

- **MAE (Mean Absolute Error)**: On average, predictions are off by ~€4.00–4.50. This is a reasonable margin for a game whose price may range from €4 to €80.
- **RMSE (Root Mean Squared Error)**: Our best model scores ~6.32. The higher RMSE relative to MAE reveals that some games (particularly mid-tier AAA titles at €40-60) are harder to predict precisely – the model underestimates their price due to limited AAA data in the dataset.
- **R^2 Score**: Best result ~0.26, meaning roughly 26% of price variance is explained by the 10 engineered features. The remaining variance is driven by brand awareness, marketing budget, and publisher reputation – factors not present in the dataset.

Dataset Curation Decisions

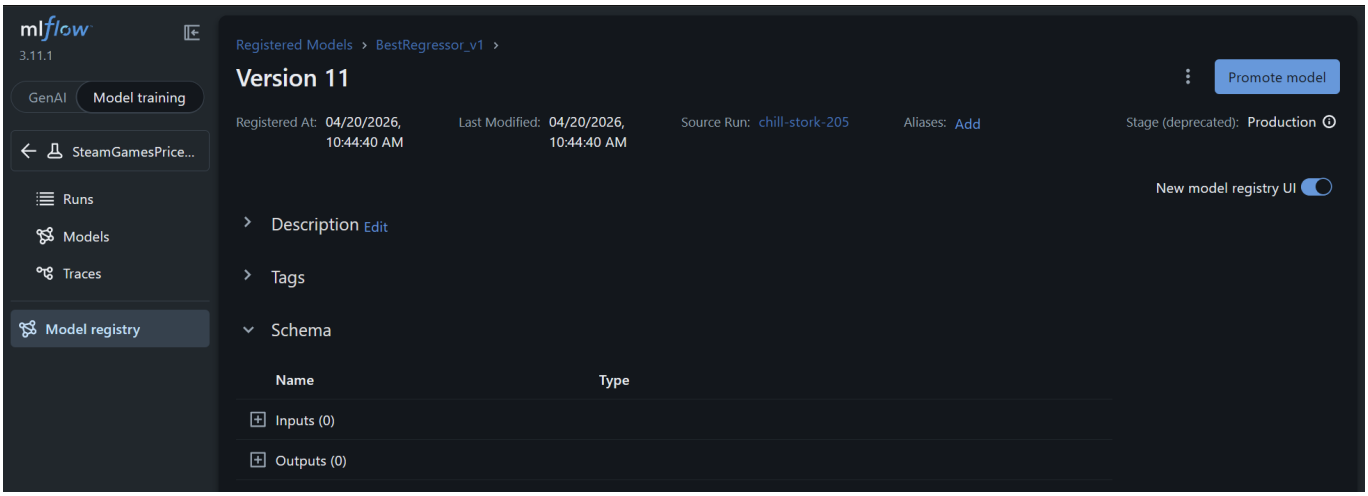
To improve model quality, strict data hygiene rules were applied before training:

- Games priced **below €4.00** and **above €80** were excluded to remove free-to-play titles and outlier bundles.
- Games priced under €6 with more than 50 achievements were removed, as these are typically achievement-farming products with artificially low prices.
- The 80/20 train-test split was stratified across three price tiers (€4–15, €15–40, €40–80) to ensure all price segments are represented in both the training and test sets.

---

4. Screenshot: Registered Model

The screenshot below confirms that `BestRegressor_v1` – Version 11 (GradientBoostingRegressor) – is successfully deployed to the **Production** stage in the MLflow Model Registry.



---

5. Bonus: Confidence Interval Interpretation (Statsmodels)

A Bonus Step was conducted using `statsmodels` (OLS) to calculate 95% confidence intervals for each regression coefficient. The results were exported as `conf_intervals.csv` and logged as an MLflow artifact on the best run.

What do these results represent? The 95% confidence intervals express the range within which the true coefficient value lies with 95% certainty. For example:

- `average_playtime`: interval is entirely above zero, meaning longer playtimes are associated with higher prices.
- `is_indie`: interval is entirely below zero, meaning indie-tagged games are statistically priced lower.
- `release_year`: positive interval, meaning more recent releases tend to be priced higher.

These intervals are also loaded directly into the Streamlit dashboard (Phase 3) and displayed as the `[95% CI: € X – € Y]` range beneath each prediction.